Data Mining: Classification

Fall 2026 (7761)

Dept. of Computer Science and Engineering

Instructor: Ting-Wei Lin

2026/11/13, Week 10

國立中興大學
NATIONAL CHUNG HSING UNIVERSITY

Practical Issues in Classification

Feature Selection and Engineering

Overfitting, Model Evaluation and Selection, and Class Imbalance

Feature Selection and Engineering

Feature Selection

- You might have collected a large number of (hundreds or thousands or even more) features
- Given *p* initial features, how do we select a small subset that is useful for prediction?
  ① Relevant features: provide information about the class label
  ② Irrelevant features: e.g., student ID for predicting GPA
  ③ Redundant features: e.g., monthly income and annual income)

Feature Engineering

- Given the initial features, how to construct more effective ones?
  - Number of daily positive cases, number of daily tests, number of daily hospitalization → weekly positive rate
- Traditionally: domain knowledge is the key
- Today, deep learning can automatically learn useful representations from raw data.

Feature Selection Methods

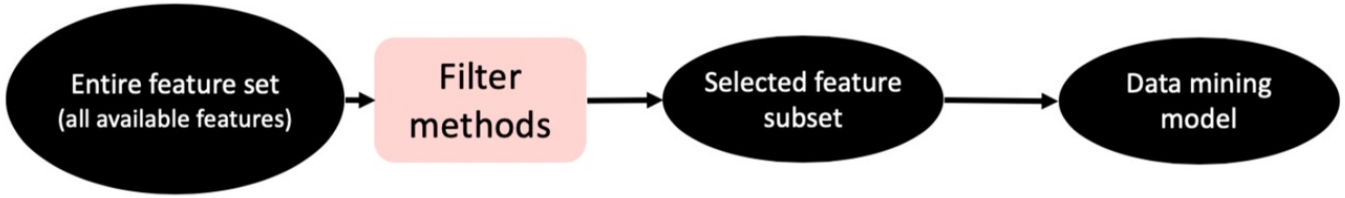
① Filter methods: score features without training a specific classifier

② Wrapper methods: search subsets using validation performance

- Repeatedly train and evaluate a classifier using the current feature subset, then update the subset.

③ Embedded methods: selection occurs during model training

Filter Methods

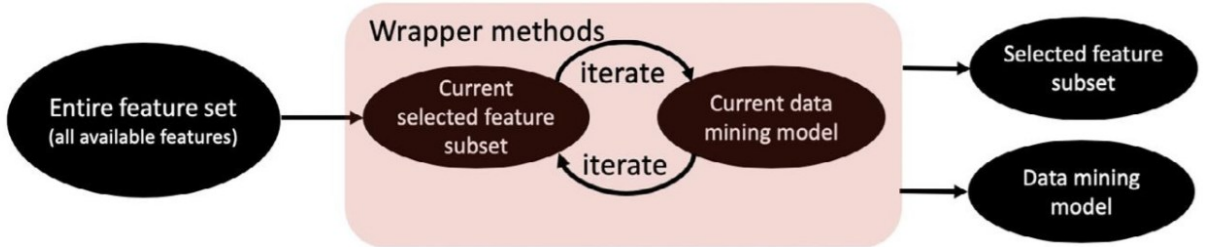


- Select good features based on some goodness measure
- Fisher score
  - Intuition: A feature is useful when its values are well separated across classes, while remaining relatively consistent within each class.
  - A higher Fisher score indicates large between-class separation and small within-class variation.

$$F_r = \frac{\sum_{k=1}^{c} n_k \left(\mu_{k,r} - \mu_r\right)^2}{\sum_{k=1}^{c} n_k \, \sigma_{k,r}^2}$$

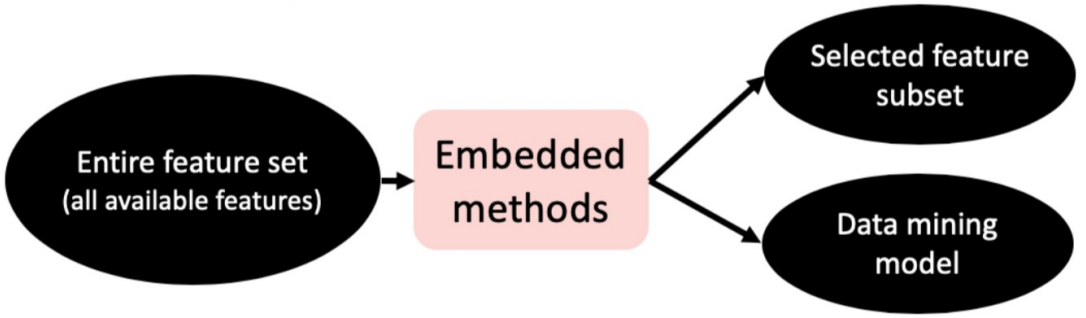
  - C is class number. $\mu_{k,r}$, $\sigma_{k,r}^2$ are mean and variance in k class of r attribute. n_k is number of the training tuple in k class

Wrapper Methods



- Evaluate candidate feature subsets by training a classifier and measuring validation performance.
  - Build a classifier based on the currently selected feature subset.
  - Based on the built classifier, it updates (e.g., add, remove, swap) the selected feature subset.
- How to search for the best feature subset
  a. Try all the possible subsets (exhaustive): $2^p - 1$
  b. Forward selection:
     - Start with an empty feature subset. At each step, add the feature that gives the largest improvement in validation performance.
  c. Backward elimination
     - Start with all features and repeatedly remove the feature whose removal hurts validation performance the least.

Embedded Methods



- Simultaneously constructs the classification model and selects the relevant features
  - Embed the feature selection step during the classification model construction step
  - Decision Tree
  - LASSO (Least Absolute Shrinkage and Selection Operator, L1 regularization)
    - Automatically decides which features to keep by forcing unimportant ones to zero during training

$$L(w) = \frac{1}{2}\sum_{i=1}^{n}(y_i - \hat{y_i})^2 + \lambda\|w\|_1 = \frac{1}{2}\sum_{i=1}^{n}(y_i - w^T x_i)^2 + \lambda\sum_{j=i}^{d}|w_j|$$

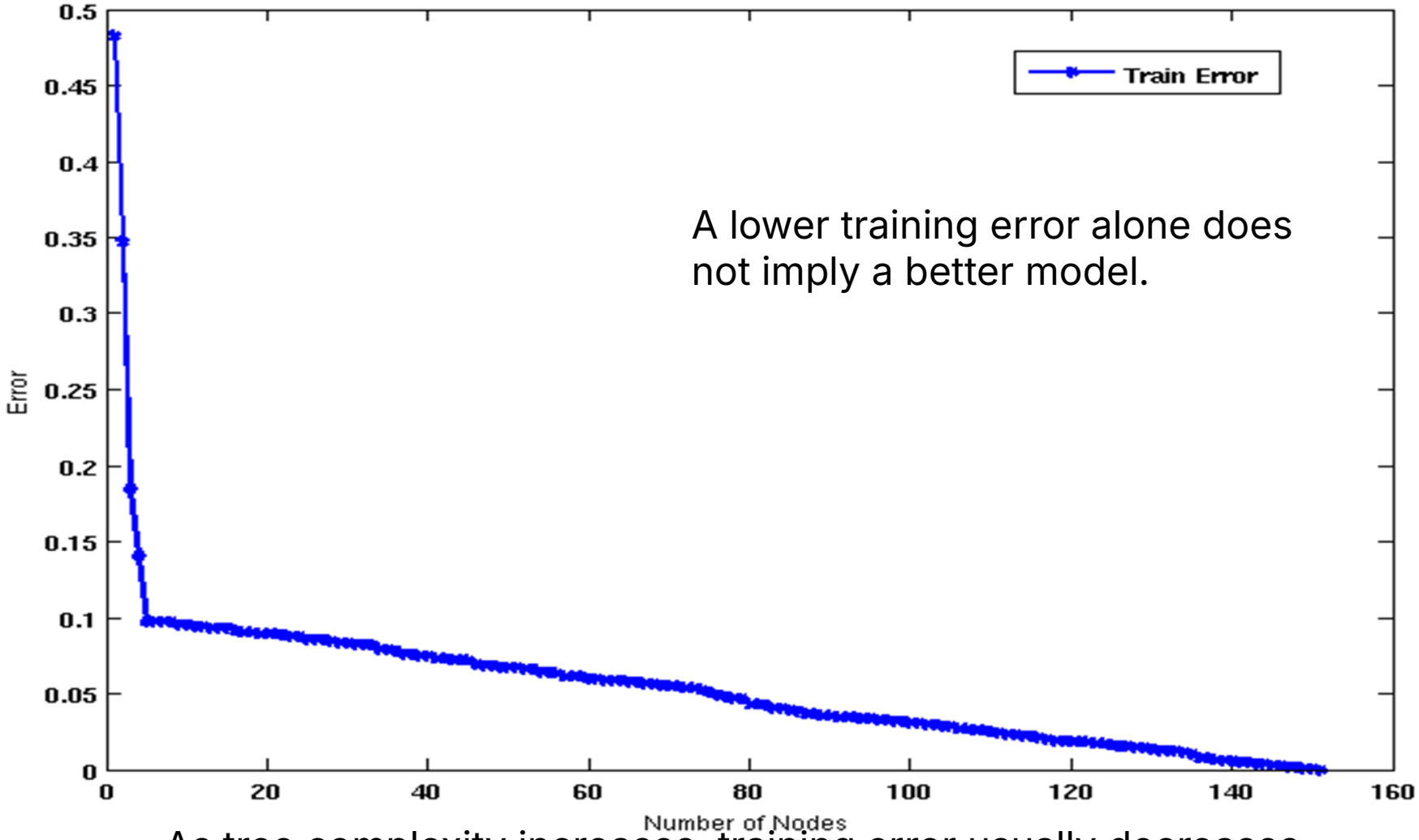
The $\|w\|_1$ penalty encourages some feature weights to become exactly zero.

Model Overfitting

Model Overfitting

- Training error: error measured on the data used to fit the model.
- Test error: expected error on unseen data. We estimate it using validation or test data.
- Overfitting: the model achieves very low training error but has poor generalization to unseen data.
  - It learns noise or accidental patterns in the training set rather than the underlying relationship.

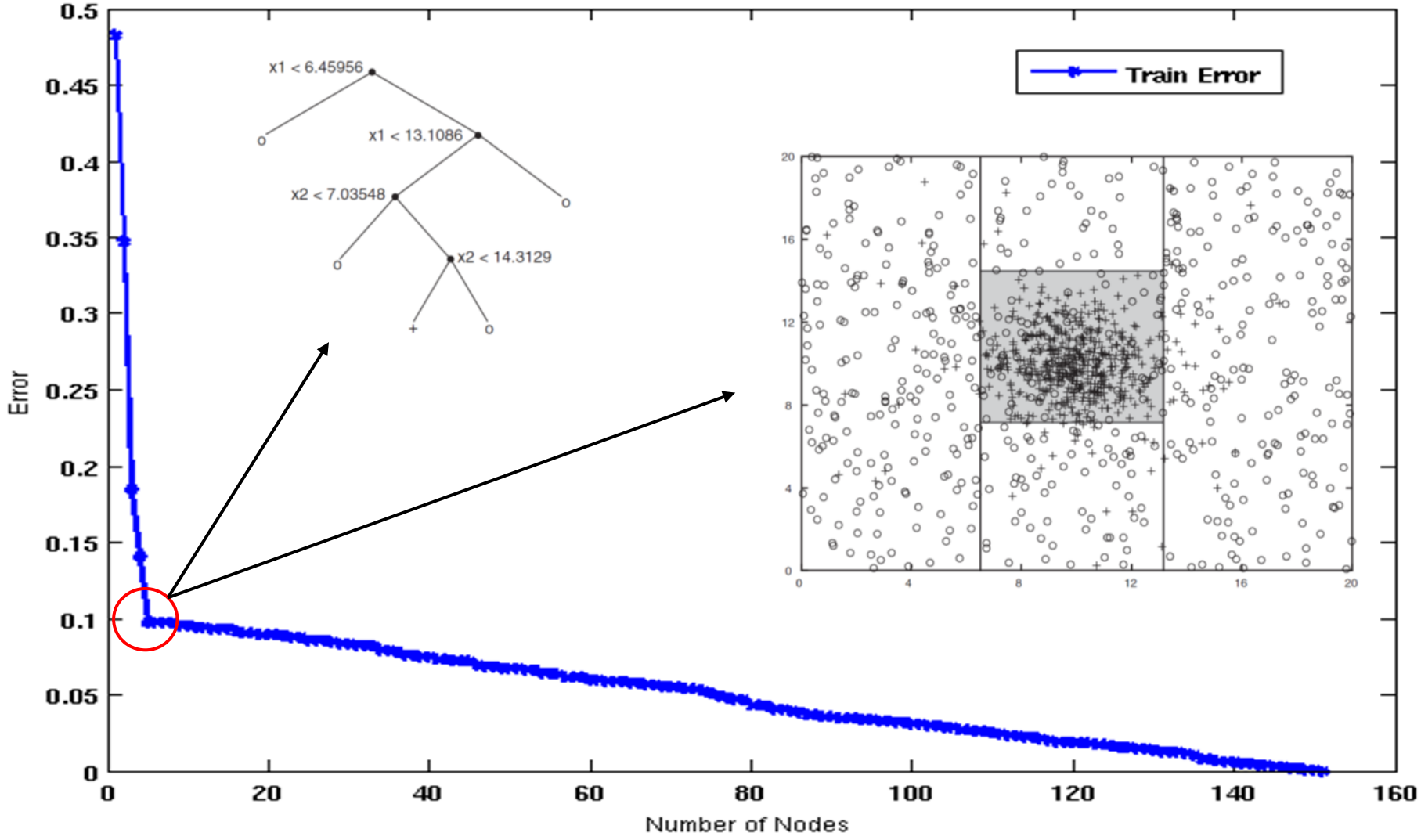
Training Error Decreases with Model Complexity



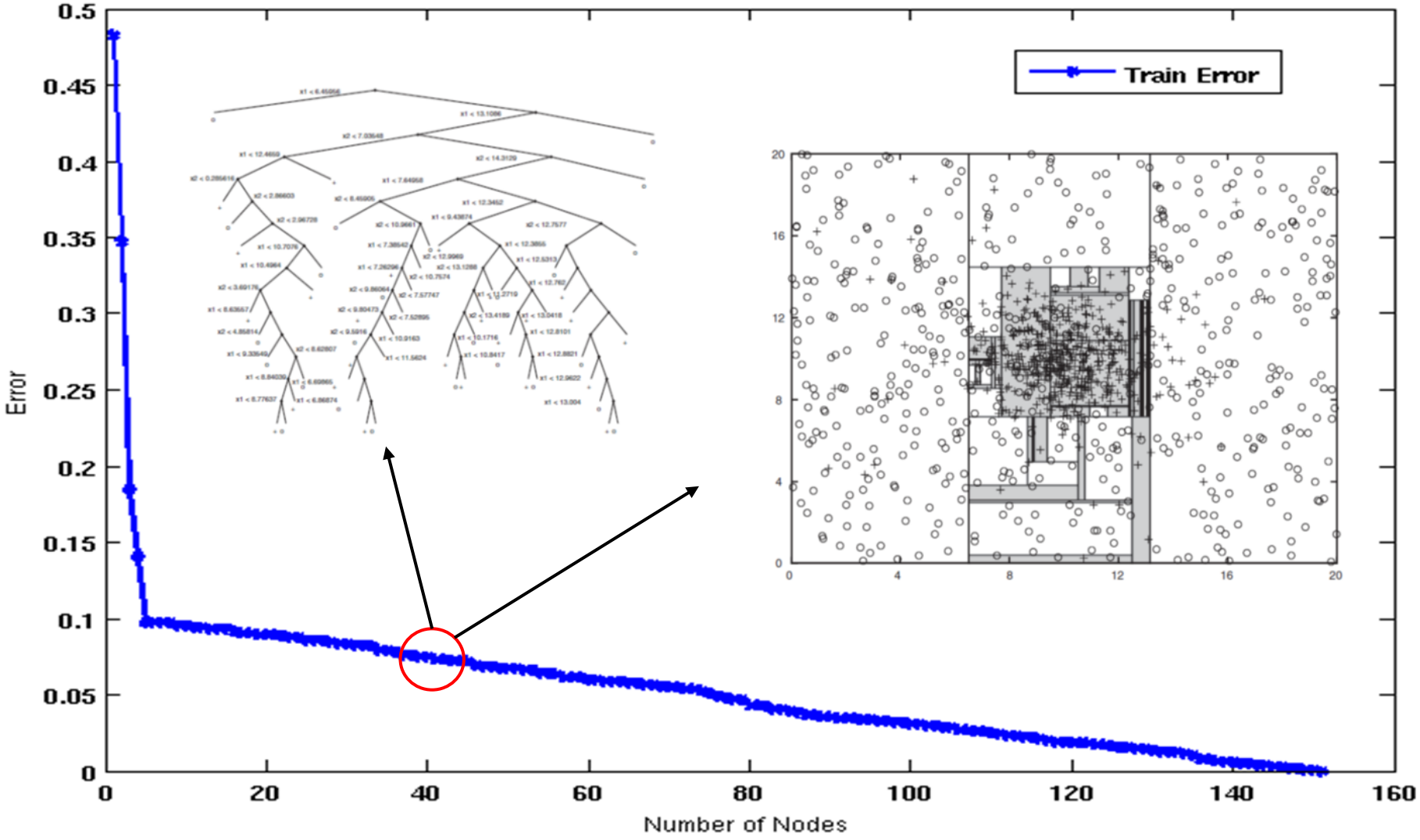
A lower training error alone does not imply a better model.

As tree complexity increases, training error usually decreases.

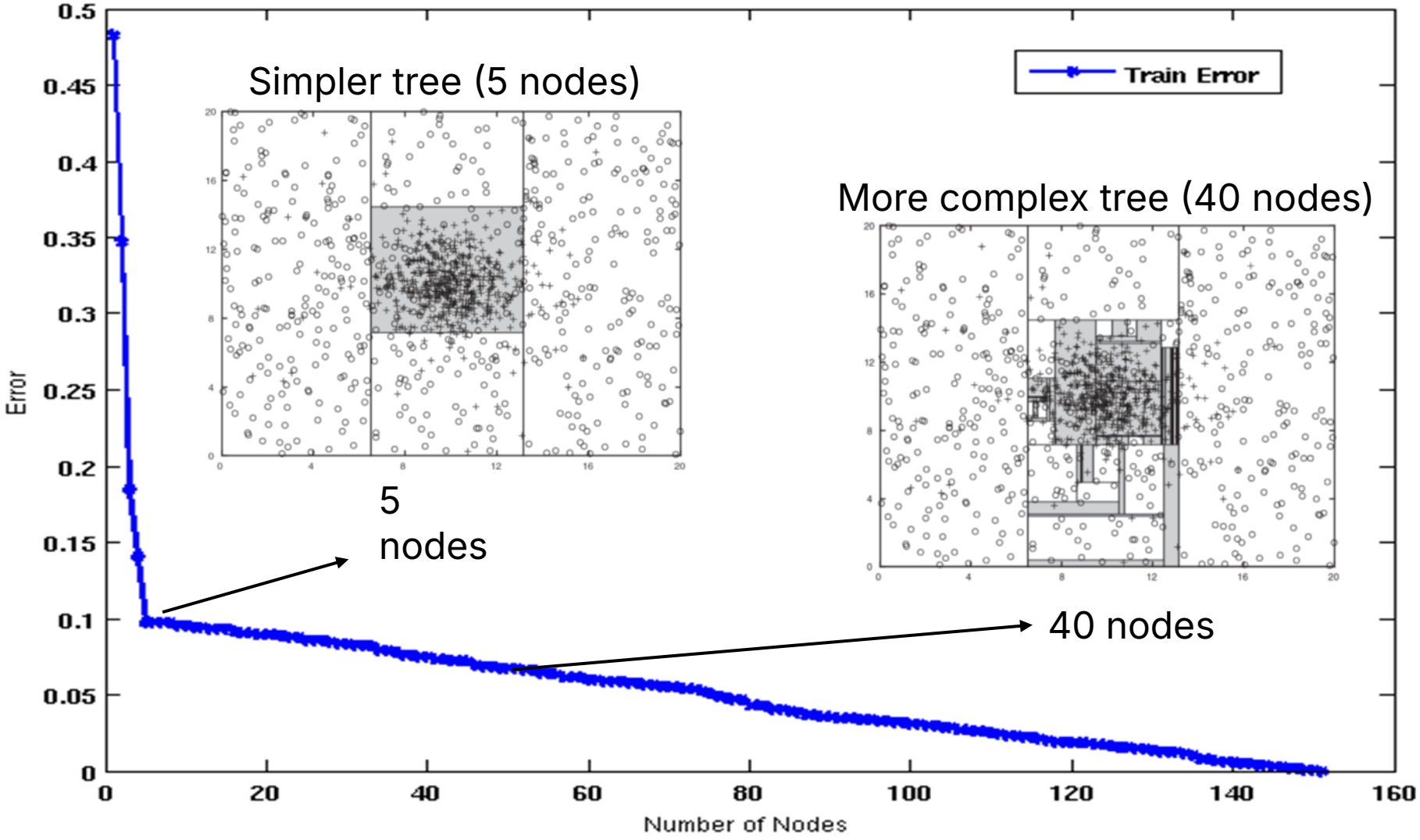
Model Overfitting (cont.)



Model Overfitting (cont.)



Model Overfitting (cont.)



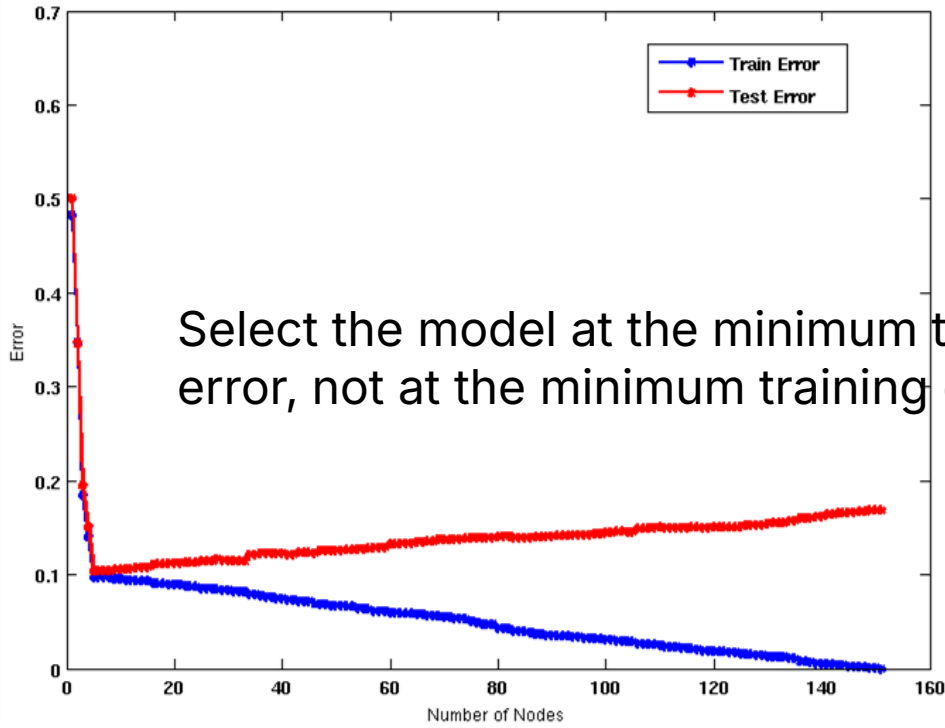
Underfitting / Good fit / Overfitting

- Underfitting: when model is too simple, both training and test errors are large
- Overfitting: when model is too complex, training error is small but test error is large
  - With more training data, the gap between training and test error often becomes smaller for a fixed model complexity.

	Model Complexity	Training Error	Test Error
Underfitting	Too Simple	High	High
Good Fit	Just Right	Low	Low
Overfitting	Too Complex	Very Low	High

Choosing Model Complexity

- As model complexity increases, training error typically decreases.
- Test error may decrease at first, then increase when the model begins to overfit.

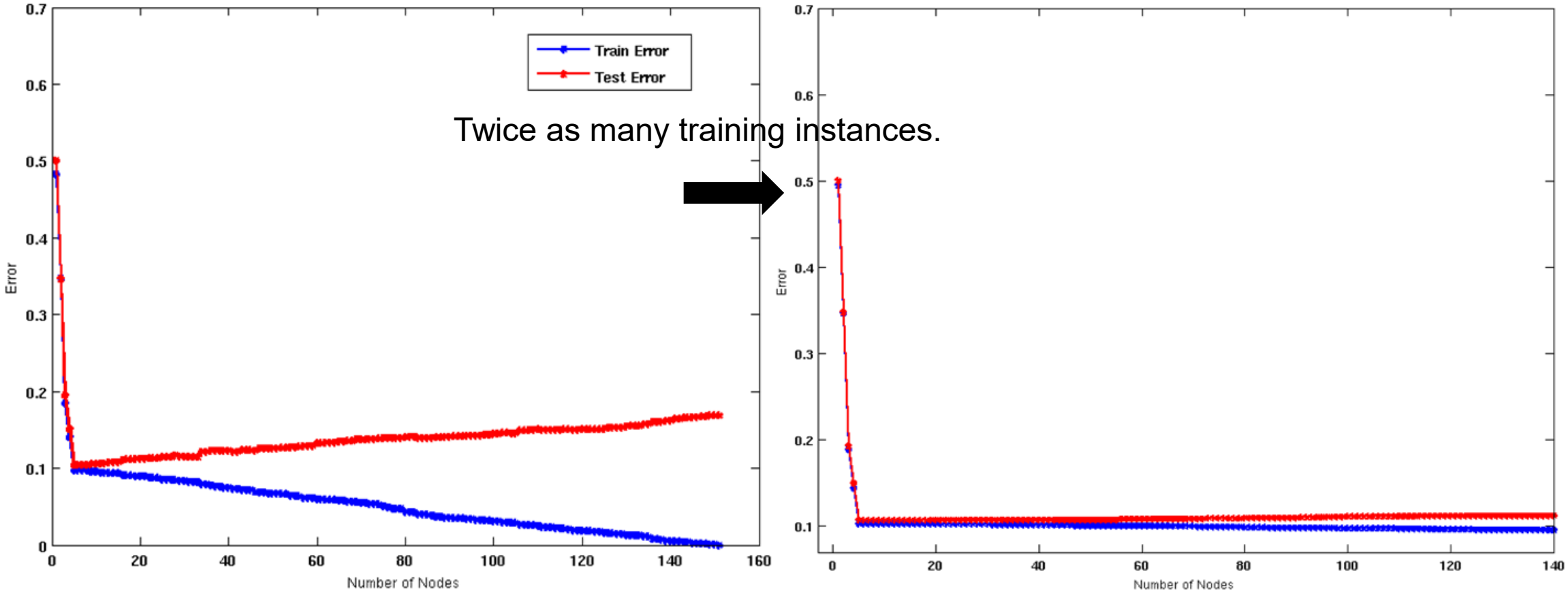


Select the model at the minimum test error, not at the minimum training error.

Reasons for Model Overfitting

- An overly complex model may capture noise or accidental patterns in the training data rather than the underlying relationship that generalizes to unseen data.
- Two of the major factors
  ① Limited training data
    - The model cannot distinguish general patterns from sampling noise.
  ② Excessive model complexity
    - The model is flexible enough to fit noise, outliers, or mislabeled examples.

Limited Training Data: Increasing the Training Set Size



Twice as many training instances.

With more representative training data, the model is less likely to fit idiosyncratic patterns of a small sample.

High model complexity

- A more complex model can represent more complicated patterns.
- If it is too complex relative to the amount of training data, it may fit noise and generalize poorly.
- The measure of complexity depends on the model:
  - Decision tree: depth or number of leaves/nodes
  - Linear model: number of features / effective degrees of freedom
  - Neural networks: number of parameters (relative to the amount and diversity of training data)

Model Evaluation and Selection

Model Evaluation and Selection

- Evaluation metrics
  - How do we measure classifier performance?
  - Accuracy, precision, recall, F1-score, ROC/AUC
- Model comparison and selection
  - How do we choose among classifiers, feature sets, and hyperparameters?
    - Use validation data and test data.

Model Evaluation and Selection

- Data splitting strategies:
  - Methods to estimate accuracy
    - Holdout method
    - Cross-validation
- Final performance assessment
  - Evaluate the final selected model once on an untouched test set.
- Use validation data for model selection and reserve the test set for final evaluation only.

Metrics for Evaluating Classifier Performance

- In binary classification, we define one class as the positive class (usually the class of primary interest) and the other as the negative class.
  - Compare the predicted label with the known (ground-truth) label for each example.

Confusion matrix

- Analyzing how well your classifier can recognize example of different classes
  - True positives (TP): the positive tuples that were correctly labeled by the classifier
  - True negatives (TN): the negative tuples that were correctly labeled by the classifier
  - False positives (FP): the negative tuples that were incorrectly labeled as positive
  - False negatives (FN): the positive tuples that were mislabeled as negative

Confusion matrix

		Prediction Class		Total
		+	-	
Actual Class	+	**True Positives (TP)**	**False Negatives (FN)**	**P=TP+FN**
	-	**False Positives (FP)**	**True Negatives (TN)**	**N=FP+TN**
Total		**P'=TP+FP**	**N'=FN+TN**	**P+N (ALL)**

Metrics from a Confusion Matrix

- Accuracy $= \frac{(\mathrm{TP} + \mathrm{TN})}{\mathrm{All}}$
  - Percentage of test set tuples that are correctly classified
- Error rate: 1 – Accuracy, $\frac{(\mathrm{FP} + \mathrm{FN})}{\mathrm{All}}$

		Prediction Class		Total
		+	-	
Actual Class	+	True Positives (TP)	False Negatives (FN)	P=TP+FN
	-	False Positives (FP)	True Negatives (TN)	N=FP+TN
Total		P'=TP+FP	N'=FN+TN	P+N (ALL)

Why Accuracy Can Be Misleading

- Significant majority of the negative class and minority of the positive class
  - In many applications, the positive class is rare.
  - Example: only 3% of patients have cancer.
    - Classifier can always answer no.
    - Accuracy = 97%
  - A high accuracy does not guarantee that the model detects the minority class.

Precision and Recall

- Accuracy $= \frac{(\mathrm{TP} + \mathrm{TN})}{\mathrm{All}}$
  - Percentage of test set tuples that are correctly classified
- Error rate: 1 – Accuracy, $\frac{(\mathrm{FP} + \mathrm{FN})}{\mathrm{All}}$
- Precision $= \frac{\mathrm{TP}}{\mathrm{TP+FP}}$
- Recall $= \frac{\mathrm{TP}}{\mathrm{TP+FN}}$

		Prediction Class		Total
		+	-	
Actual Class	+	True Positives (TP)	False Negatives (FN)	P=TP+FN
	-	False Positives (FP)	True Negatives (TN)	N=FP+TN
Total		P'=TP+FP	N'=FN+TN	P+N (ALL)

F1-score

- F1-score (F1-measure):

$$\mathrm{F} = \frac{2 \times \mathrm{precision} \times \mathrm{recall}}{\mathrm{precision} + \mathrm{recall}}$$

- The weighted measure of precision & recall

$$\mathrm{F} = \frac{(\beta^2 + 1) \times \mathrm{precision} \times \mathrm{recall}}{\beta^2 \times \mathrm{precision} + \mathrm{recall}}$$

  - $\beta = 1$: precision and recall have equal importance
  - $\beta > 1$: more focus on recall
  - $\beta < 1$: more focus on precision

Metrics from a Confusion Matrix

- True Positive Rate (sensitivity, recall) $= \frac{\mathrm{TP}}{\mathrm{TP+FN}}$
- True Negative Rate (specificity) $= \frac{\mathrm{TN}}{\mathrm{FP+TN}}$
- False Positive Rate $= \frac{\mathrm{FP}}{\mathrm{FP+TN}}$
- False Negative Rate $= \frac{\mathrm{FN}}{\mathrm{TP+FN}}$

		Prediction Class		Total
		+	-	
Actual Class	+	True Positives (TP)	False Negatives (FN)	P=TP+FN
	-	False Positives (FP)	True Negatives (TN)	N=FP+TN
Total		P'=TP+FP	N'=FN+TN	P+N (ALL)

Confusion matrix: Exercise

- Accuracy $= \frac{\text{TP} + \text{TN}}{\text{All}} = (90 + 9560)/10000 = 96.50\%$
- Error rate $= 3.50\%$
- Precision $= \frac{\text{TP}}{\text{TP+FP}} = 90/(90 + 140) = 90/230 = 39.13\%$
- Recall (Sensitivity) $= \frac{\text{TP}}{\text{TP+FN}} = 90/(90 + 210) = 90/300 = 30.00\%$
- Specificity $= \frac{\text{TN}}{\text{TN+FP}} = 9560/(140 + 9560) = 98.56\%$
- F1 $= 2p \times r/(p + r) = 2 \times 39.13\% \times 30.00\%/(39.13\% + 30\%) = 33.96\%$

Actual Class\Predicted class	cancer = yes	cancer = no	Total
cancer = yes	**90**	**210**	300
cancer = no	**140**	**9560**	9700
Total	230	9770	10000

Confusion matrix: Exercise

- Let's think about:
  - The model is good or bad?
  - Improve recall? Or precision?
- Despite 96.5% accuracy, the model misses
  - $210/300 = 70\%$ of patients who actually have cancer. $(\text{recall} = 30\%)$ of patients who actually have cancer.
- For a system, should we prioritize higher recall, higher precision, or a balance between them?

A		Prediction Class	
		+	-
Actual Class	+	**10**	**40**
	-	**10**	**40**

Precision = 0.5
Recall (TPR) = 0.2
F-measure = 0.28
FPR = 0.2

B		Prediction Class	
		+	-
Actual Class	+	**25**	**25**
	-	**25**	**25**

Precision = 0.5
Recall (TPR) = 0.5
F-measure = 0.5
FPR = 0.5

C		Prediction Class	
		+	-
Actual Class	+	**40**	**10**
	-	**40**	**10**

Precision = 0.5
Recall (TPR) = 0.8
F-measure = 0.61
FPR= 0.8

Comparing Three Binary Classifiers

ROC Curve: Comparing TPR and FPR

- Receiver Operating Characteristic Curve: a visual tool for comparing classification models.
  - An ROC curve visualizes the trade-off between the true positive rate (TPR) and false positive rate (FPR) as the decision threshold changes.
  - TPR = 0, FPR = 0: predicts each instance to be negative
  - TPR = 1, FPR=1: predicts each instance to be positive
  - TPR=1, FPR=0: ideal model
  - A better model achieves a higher TPR for the same FPR; its curve is closer to the upper-left corner.

A	
TP=63	FN=37
FP=28	TN=72

$$TPR = 0.63$$
$$FPR = 0.28$$

B	
TP=77	FN=23
FP=77	TN=23

$$TPR = 0.77$$
$$FPR = 0.77$$

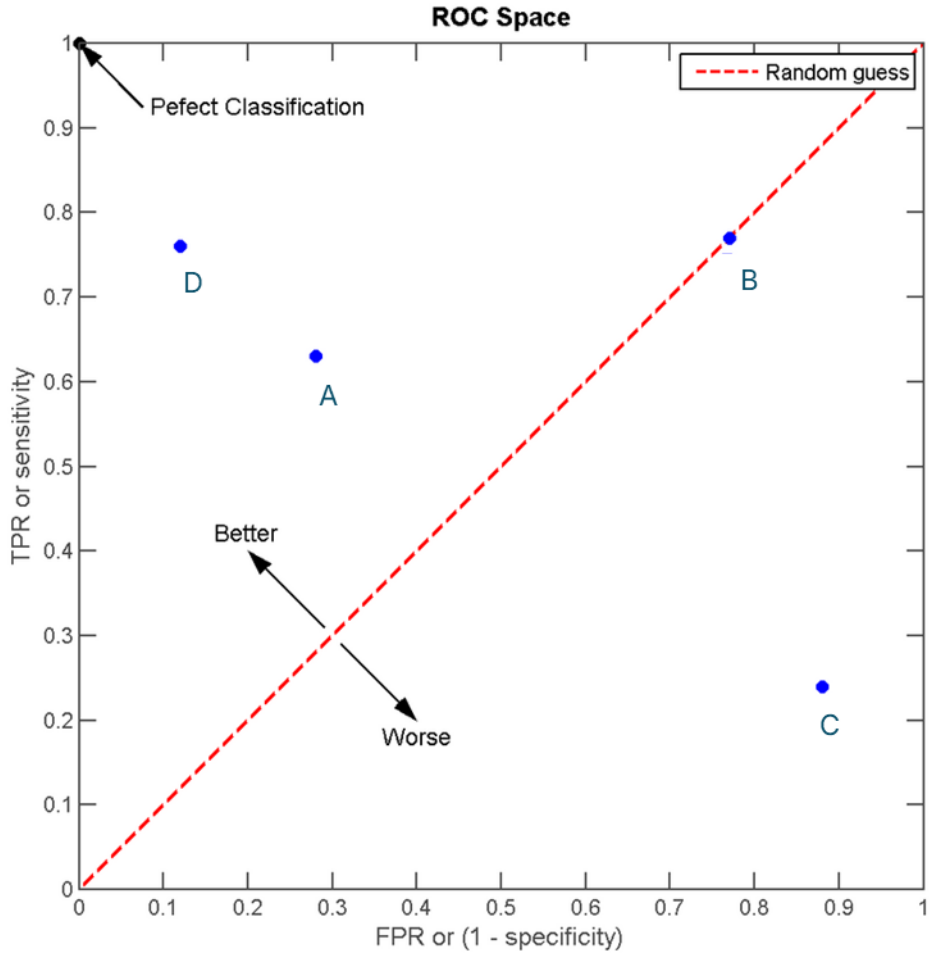
C	
TP=24	FN=76
FP=88	TN=12

$$TPR = 0.24$$
$$FPR = 0.88$$

D	
TP=76	FN=24
FP=12	TN=88

$$TPR = 0.76$$
$$FPR = 0.12$$

Different decision threshold:
A, B, C, and D



ROC Space

Source:
https://en.wikipedia.org/wiki/Receiver_operating_characteristic

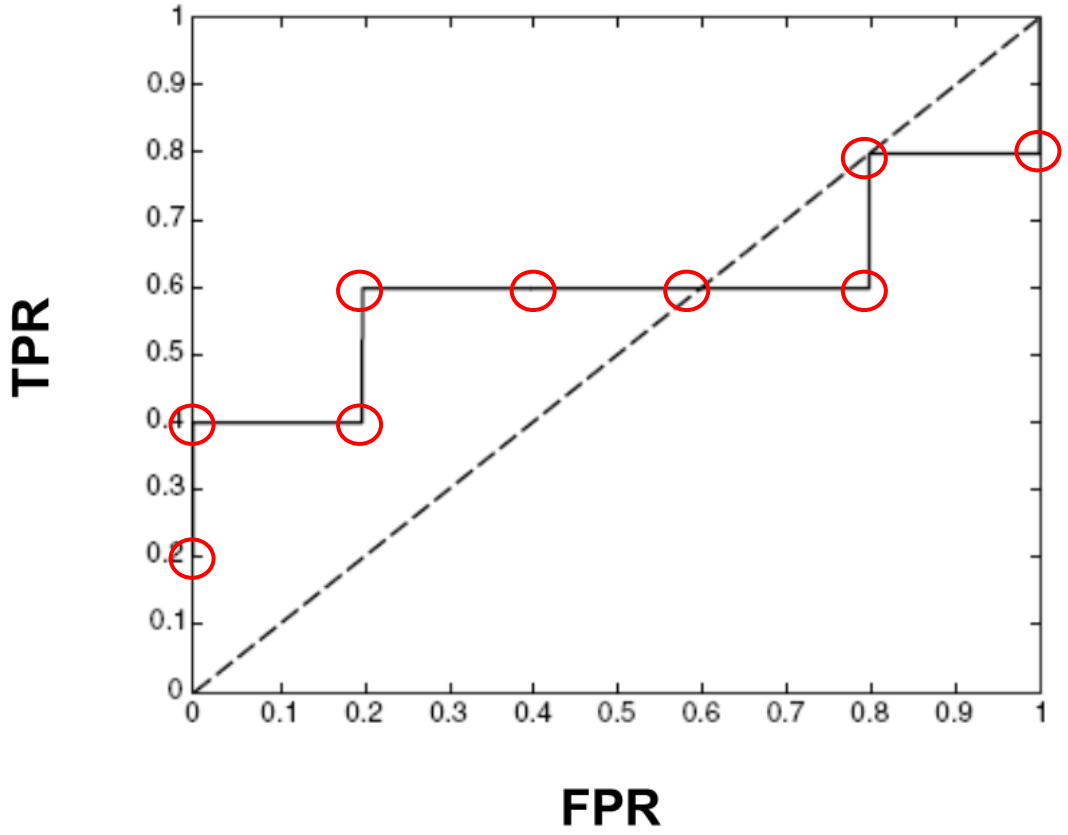
Constructing an ROC Curve

- A classifier must output a score $s(x)$ (e.g., probability or decision score) that ranks instances.
  - Higher score indicates that an instance is more likely to be positive.
- Choose a threshold t
  - Predict positive when $s(x) \geq t$.
  - Otherwise, predict negative.

Constructing an ROC Curve

- Steps:
  ① Set t above the highest score: predict all instances as negative. (TPR= FPR=0)
  ② Lower t to include the highest-ranked instance: predict the top 1 instance as positive and all remaining instances as negative.
  ③ Continue lowering t: predict the top 2, top 3, … instances as positive.
  ④ Repeat accordingly until the bottom ranked data (TPR=FPR=1)
  ⑤ Plot each (FPR,TPR) pair and connect the points.

Class	+	−	+	−	−	−	+	−	+	+	
	0.25	0.43	0.53	0.76	0.85	0.85	0.85	0.87	0.93	0.95	1.00
TP	5	4	4	3	3	3	3	2	2	1	0
FP	5	5	4	4	3	2	1	1	0	0	0
TN	0	0	1	1	2	3	4	4	5	5	5
FN	0	1	1	2	2	2	2	3	3	4	5
TPR	1	0.8	0.8	0.6	0.6	0.6	0.6	0.4	0.4	0.2	0
FPR	1	1	0.8	0.8	0.6	0.4	0.2	0.2	0	0	0



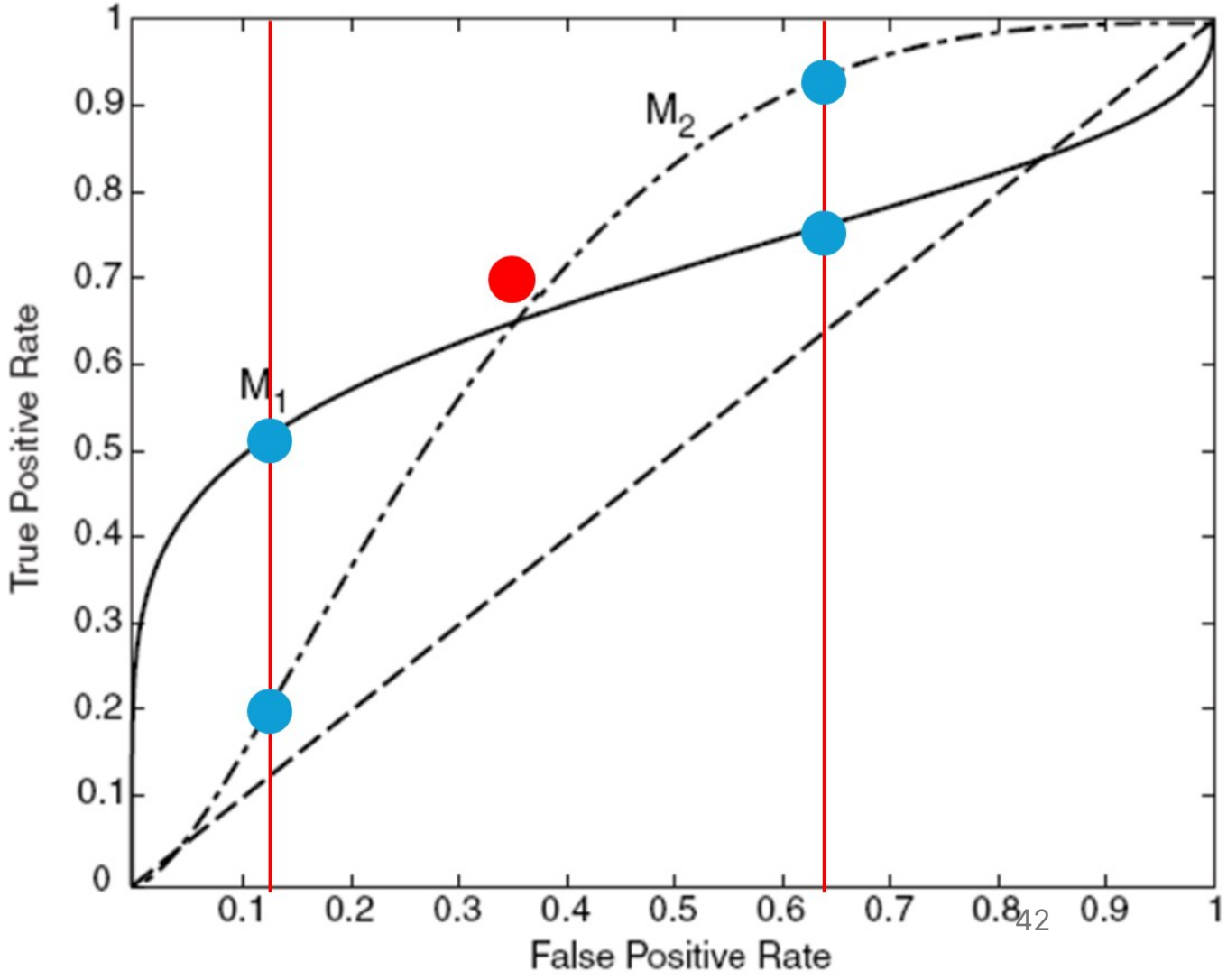
AUC (Area Under ROC Curve)

- Measure how well a model ranks positive examples above negative examples across all possible thresholds.
  - If the model is perfect, AUC would equal 1
  - If the model simply perform random guessing, AUC would equal 0.5
  - A larger AUC indicates better overall ranking performance, although the preferred model may still depend on the relevant FPR range.

Choosing a Model Depends on the Operating Region

At low FPR, M_1 has higher TPR.
At high FPR, M_2 has higher TPR.

Fraud detection vs Disease screening
- Accept more false positive for higher recall?



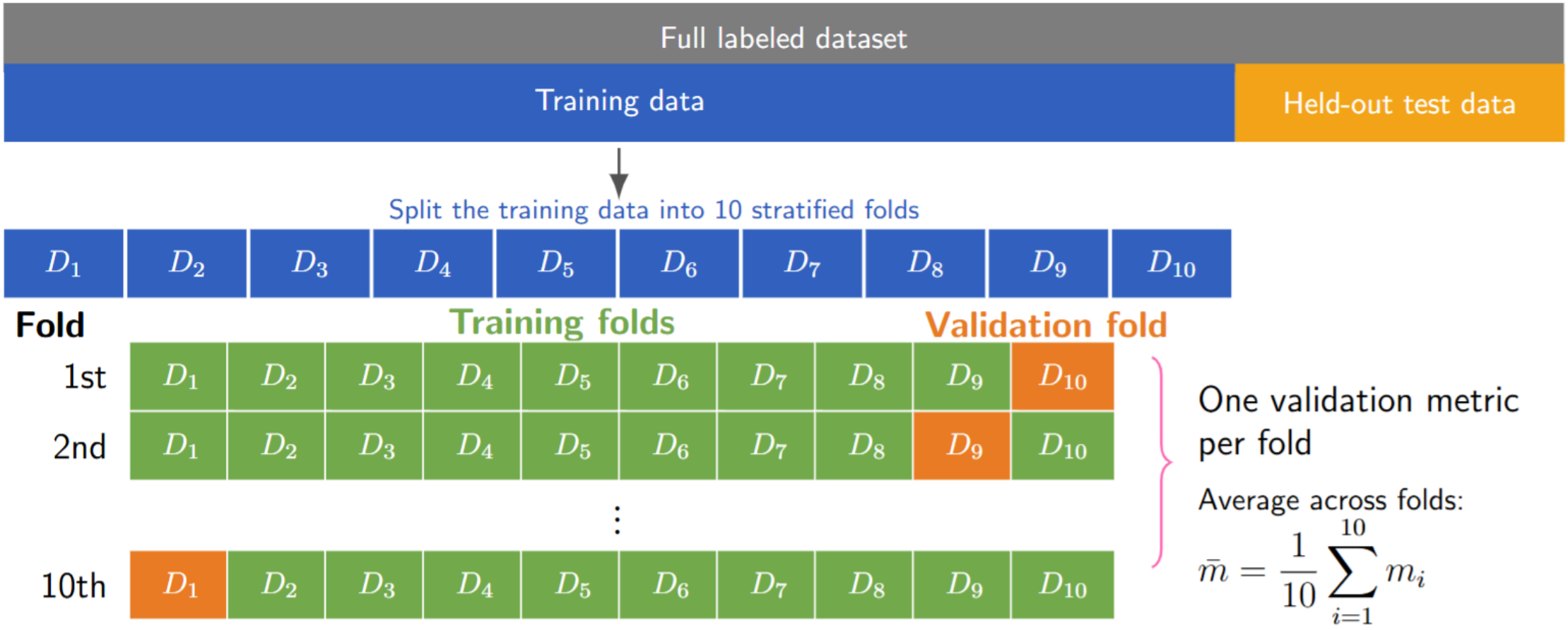
Classifier Evaluation: Holdout Method

- Randomly split the data into two disjoint subsets:
  - Training set (e.g., 2/3): fit the model
  - Test set (e.g., 1/3): evaluate the model
- Repeat holdout k times, Report the mean performance across repetitions.

Classifier Evaluation: Cross-Validation

- k-fold cross-validation
  - Randomly partition the training data into k mutually exclusive approximately equal-sized folds.
  - Repeat k times: use one fold for validation and the remaining $k-1$ folds for training.
  - Average the validation metric across the k folds.
  - Common choices: $k = 5$ or $k = 10$.

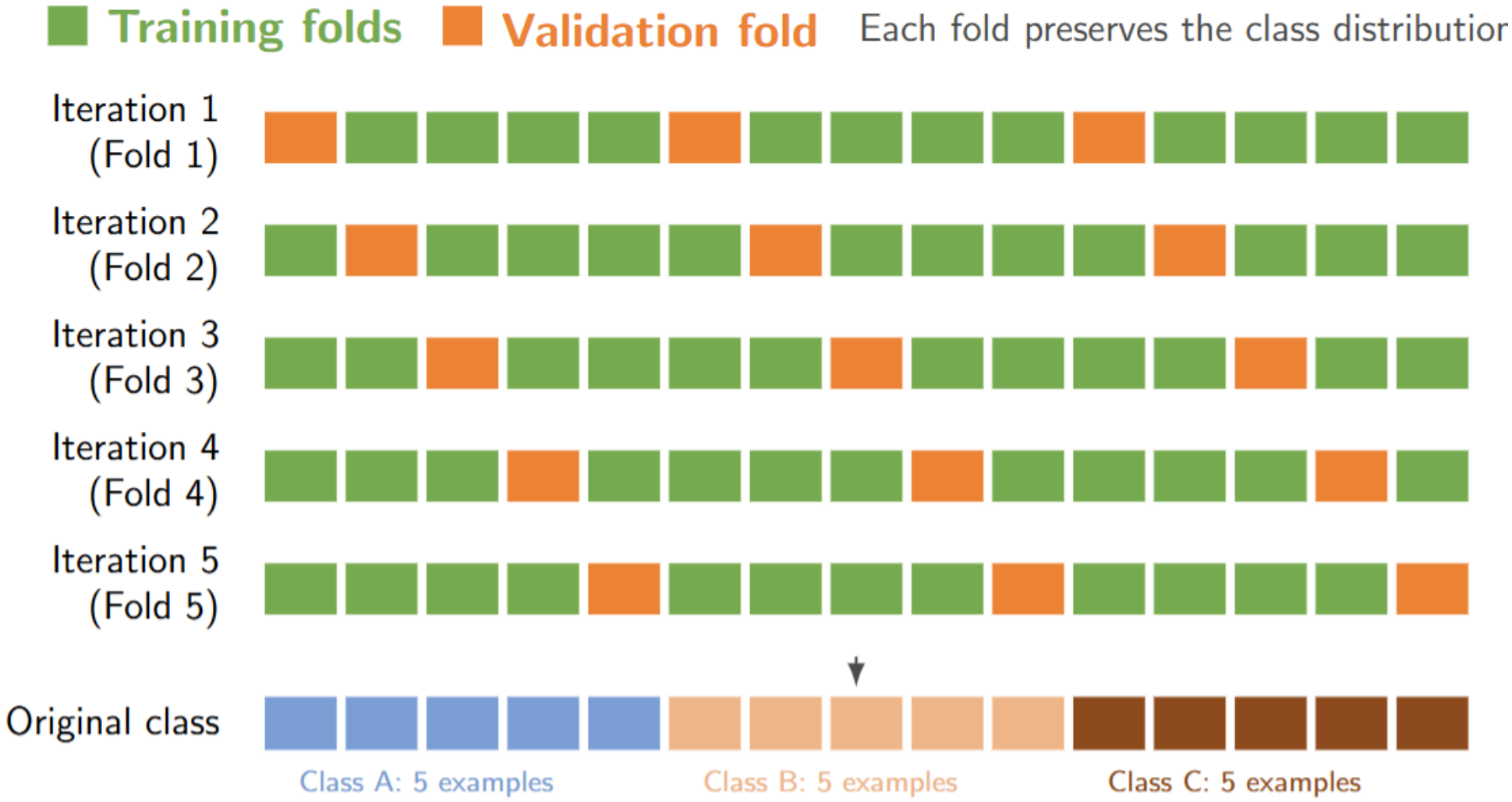
k-fold cross-validation



Classifier Evaluation: Cross-Validation

- Leave-one-out (LOOCV)
  - $k = n$, where n is number of instances, usually for small sized data.
  - Each fold contains one instance.
- Stratified cross-validation
  - Each fold has approximately the same class proportions as the full training set.

Stratified cross-validation



Handling Class-Imbalanced Data: Data-Level Methods

- In many real-world problems, one class is much rarer than the others.
  - Fraud detection, disease screening, network intrusion detection.
- Resampling methods are applied only to the training data:
  1. Random oversampling: duplicate minority-class examples.
  2. Random undersampling: remove some majority-class examples.
  3. Synthetic oversampling (e.g., SMOTE): create new minority-class examples.

Handling Class-Imbalanced Data: Algorithm-Level Methods

① Threshold moving:

- Lower the positive-class decision threshold to increase recall and reduce costly false negatives.
- Choose the threshold using validation data or cross-validation.

② Class-weighted learning:

- Assign a larger penalty to misclassifying minority-class examples.
- Set a larger weight for the positive/minority class.

③ Ensemble methods

- Balanced random forest or boosting methods designed for imbalanced data.

Summary: Building a Classification Model

- Prepare the data
  - Handle missing values and outliers.
  - Transform, create, and select useful features.
- Choose candidate models
  - Train multiple algorithms or model configurations.
- Tune using validation or cross-validation
  - Select hyperparameters, features, and decision thresholds.

Summary: Evaluating Classification Models Responsibly

- Use metrics appropriate for the task
  - Accuracy, precision, recall, F-score, ROC curve, and AUC.
- Handle class imbalance
  - Resampling: oversampling, undersampling, or SMOTE.
  - Algorithm-level methods: class weights, cost-sensitive learning, or threshold moving.
- A good classifier is not simply the one with the highest accuracy. It is the one that generalizes well and meets the task's error-cost requirements.